به نام خدا

استاد: خانم رضایی | گزارش کار آزمایشگاه سیستم دیجیتال ۲ (جلسه اول) | علی سپهرنیا (۹۶۱۲۱۰۳۸)

۱- ساخت شمارنده باینری توسط میکروکنترلر بر روی ۴ عدد LED

می خواهیم با اتصال ۴ بیت از پورت B به ۴ عدد LED به رنگ قرمز یک شمارنده باینری صعودی ۴ بیتی با زمان تناوب ۵۰۰ یا ۲۵۰ میلی ثانیه بر روی ۴ عدد LED توسط میکروکنترلر ایجاد نماییم.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. در تب Ports پورت B را انتخاب میکنیم و بیت های کم ارزش آن یعنی Bit 0…3 را روی حالت Out یا همان ۱ میگذاریم تا بتوانیم روی آن بنویسیم و LED ها را به آن وصل کنیم. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. در تابع main متغیر int i را برای استفاده در حلقه for تعریف میکنیم. در حلقه بینهایت یک حلقه for مینویسیم که از i=0 تا i<=15 بشمارد. در داخل حلقه خط PORTB=i را مینویسیم تا مقدار شمارش شده را در پورت B بنویسد. سپس delay_ms(500) را اضافه میکنیم تا با هر بار شمارش 500ms صبر کند و بعد به شماره بعدی برود. به این شکل تغییرات شمارنده قابل مشاهده است. برای استفاده از تابع delay_ms کتابخانه delay را لازم داریم. در بالای برنامه بعد از خط #include <mega16.h> عبارت #include <delay.h> را اضافه میکنیم تا کتابخانه delay فراخوانی شود. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که با هر بار اجرای حلقه بینهایت، حلقه for تکرار میشود. حلقه for نیز هر بار از شماره ۰ تا ۱۵ را می شمارد و در هربار شمارش این مقدار را به پورت B میفرستد و 500ms صبر میکند. بدین ترتیب میکروکنترلر مدام باینری اعداد ۰ تا ۱۵ را در پورت B مینویسد. با وصل کردن ۴ LED به این پورت میتوانیم شمارش را مشاهده کنیم.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم.کامپونت های LED اکتیو و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. یک عدد میکروکنترلر و چهار عدد LED به شماتیک اضافه میکنیم. یک سر LED ها را به ترتیب از راست به چپ به بیت های کم ارزش پورت B وصل میکنیم. سمت دیگر LED ها را به زمین متصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که LED ها اعداد ۰ تا ۱۵ را به صورت باینری می شمارد و مدام این شمارش را تکرار میکند.

۲- اسکن کلید های پوش و کشویی

می خواهیم با اتصال یک کلید از بلوک PUSH BUTTON SWITCH به پین اول از پورت D وضعیت ۰ و یا ۱ بودن کلید را بررسی و با اتصال پین اول از پورت B به یک LED وضعیت کلید را مشخص نماییم.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. بقیه تنظیمات را تغییر نمیدهیم. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. رجیستر DDRD را برابر 0x00 قرار میدهیم تا به عنوان ورودی تعریف شود. همچنین PORTD را برابر 0x01 قرار میدهیم تا بیت ۰ آن در وضعیت مقاومت بالاکش (pull-up) قرار بگیرد. با اینکار میتوانیم به راحتی باز یا بسته بودن کلید را بررسی کنیم. رجیستر DDRB را برابر 0x01 قرار میدهیم تا بیت ۰ آن در وضعیت خروجی قرار بگیرد و بتوانیم LED را با آن کنترل کنیم.
چون قرار است کلید به صورت (pull-up) بسته شود بنابراین در حالت عادی مقدار ۱ و در حالت فشرده شده مقدار ۰ را بر میگرداند. در حلقه بینهایت جمله شرطیif-else مینویسیم. تعریف میکنیم اگر PIND.0==0 یعنی اگر مقدار خوانده شده از بیت ۰ پورت D برابر صفر بود، مقدار ۱ و در غیر اینصورت مقدار ۰ را در بیت ۰ پورت B بنویسد (PORTB.0=1 یا PORTB.0=0). از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که با هر بار اجرای حلقه بینهایت، شرط بررسی می شود. اگر کلید فشرده شده بود (PIND.0==0) LED را روشن میکند (PORTB.0=1) و در غیر اینصورت LED را خاموش میکند (PORTB.0=0). بدین ترتیب در حالت عادی LED خاموش است و هنگامی که کلید فشرده می شود تا هنگامی که رها شود LED روشن میماند.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم.کامپوننت های LED اکتیو، BUTTON اکتیو و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. یک عدد میکروکنترلر، یک عدد LED و یک عدد BUTTON به شماتیک اضافه میکنیم. یک سر LED را به بیت ۰ پورت B و سر دیگر آن را به زمین وصل میکنیم. یک سر کلید را به پین ۰ پورت D و سر دیگر آن را به زمین وصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که LED خاموش است و هر وقت دکمه فشرده می شود LED روشن میشود و تا زمانی که رها نشده روشن میماند.

۳- ساخت شمارنده از صفر تا نه با استفاده از ۷-سگمنت

برنامه ای بنویسید که عدد ۰ تا ۹ را در ۷سگمنت نمایش دهد و به صورت برعکس، برای اینکار در ابتدا کدهای HEX معادل هر عدد را در ۷ سگمنت بدست آورید و پورت خروجی مورد نظر را با این مقادیر مقداردهی کنید.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. بقیه تنظیمات را تغییر نمیدهیم. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. رجیستر DDRD را برابر 0xff قرار میدهیم با اینکار همه ی پین های پورت D به عنوان خروجی تعریف می شود. برای کار با سون سگمنت کد هگز ارقام ۰ تا ۹ را بدست می آوریم. مثلا برای عدد 0 باید همه ی LED ها به جز LED g روشن باشد، چون از سون سگمنت کاتد مشترک استفاده میکنیم (فعال سازی با ۱)، کد هگز رقم 0 برابر 0b00111111=0x3f می شود. برای ارقام 1 تا 9 نیز به همین صورت عمل میکنیم و مقادیر زیر به دست می آید:

0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f

در تابع main آرایه sevenseg از نوع int را تعریف میکنیم و کدهای هگز فوق را در آن ذخیره میکنیم. بدین ترتیب اندیس i ام آرایه کد هگز عدد i را نشان میدهد. متغیر i را نیز برای حلقه شمارنده تعریف میکنیم. برنامه باید یکبار به صورت صعودی از ۰ تا ۹ و بعد به صورت نزولی از ۹ تا ۰ بشمارد. بنابراین در حلقه بینهایت دو حلقه for مینویسیم. به طوری که حلقه اول از i=0 تا i<=9 و حلقه دوم از i=8 تا i>=1 بشمارد. علت شمارش از ۸ تا ۱ در حلقه دوم این است که شماره ۹ و ۰ یکبار در حلقه اول شمارش می شوند. سپس در هر دو حلقه PORTD را برابر sevenseg[i] قرار میدهیم تا کد هگز هر عدد شمارنده در پورت D قرار بگیرد. delay_ms(500) را نیز در خط بعد در هر دو حلقه اضافه میکنیم تا بعد از هر شمارش 500ms صبر کند. به این شکل تغییرات شمارنده قابل مشاهده است. برای استفاده از تابع delay_ms کتابخانه delay را لازم داریم. در بالای برنامه بعد از خط #include <mega16.h> عبارت #include <delay.h> را اضافه میکنیم تا کتابخانه delay فراخوانی شود. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که با هر بار اجرای حلقه بینهایت یک بار اعداد ۰ تا ۹ به صورت صعودی و بعد یکبار به صورت نزولی شمارش می شوند. کد هگز معادل شمارنده در هر شمارش روی پورت D که متصل به سون سگمنت است نوشته می شود. بدین ترتیب اعداد ۰ تا ۹ به صورت صعودی و بعد نزولی روی سون سگمنت نمایش داده می شوند و این چرخه مدام تکرار می شود.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم.کامپوننت های سون سگمنت کاتد مشترک و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. یک عدد میکروکنترلر، و یک عدد سون سگمنت به شماتیک اضافه میکنیم. پایه های a تا g سون سگمنت را به ترتیب به بیت های ۰ تا ۷ پورت D متصل میکنیم. پایه مشترک سون سگمنت را به زمین وصل میکنیم. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم اعداد ۰ تا ۹ و سپس برعکس روی سون سگمنت شمارش می شوند و این چرخه ادامه پیدا میکند.

پیوست ها:

آزمایش ۱)

G:\Personal\Uni\Micro Lab\Micro\Az1\Az1.c

Notes | Az1.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session1-Experiment1
Version :
Date    : 10/2/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <delay.h>

// Declare your global variables here

void main(void)
{
// Declare your local variables here
int i;

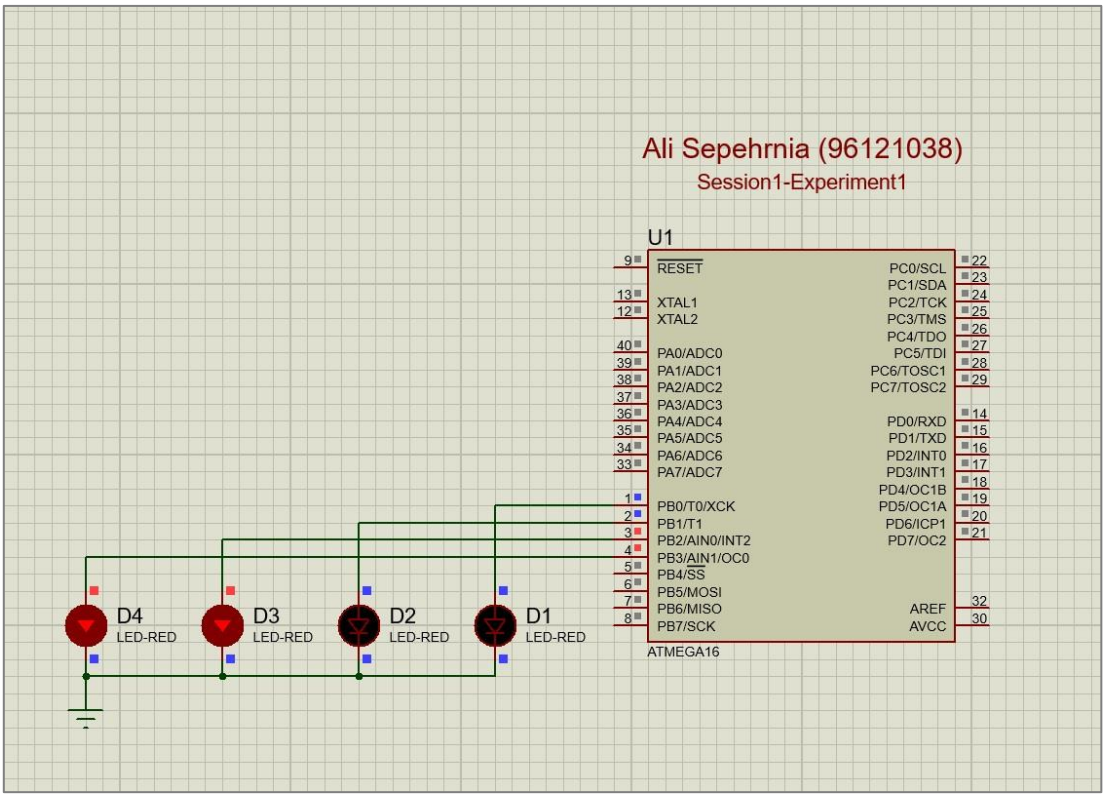
// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=Out Bit2=Out Bit1=Out Bit0=Out
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) | (1<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

while (1)
      {
          // Place your code here
          for(i=0;i<=15;i++)
          {
            PORTB=i;
            delay_ms(500);
          }
      }
}
```



آزمایش ۲)

G:\Personal\Uni\Micro Lab\Micro\Az1-2\Az1-2.c

Notes | Az1-2.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session1-Experiment2
Version :
Date    : 10/2/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>

// Declare your global variables here

void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
```

G:\Personal\Uni\Micro Lab\Micro\Az1-2\Az1-2.c

Notes | Az1-2.c

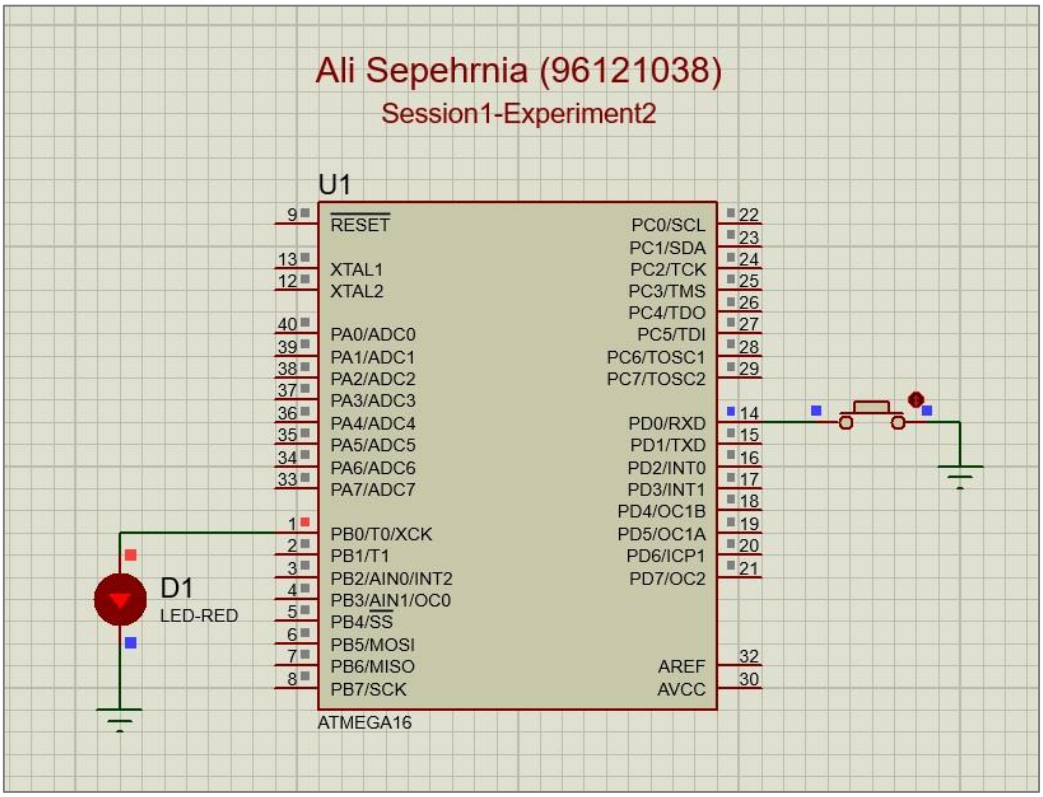
```
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=Out Bit0=In
DDRB=0x01;
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=0x00;
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=0x01;

while (1)
      {
        if(PIND.0==0)
            PORTB.0=1;
        else
            PORTB.0=0;
      }
}
```



آزمایش ۳)

G:\Personal\Uni\Micro Lab\Micro\Az1-3\Az1-3.c

Notes | Az1-3.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session1-Experiment3
Version :
Date    : 10/2/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <delay.h>

// Declare your global variables here

void main(void)
{
// Declare your local variables here
int sevenseg[] = {0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};
int i;

// Input/Output Ports initialization
```

G:\Personal\Uni\Micro Lab\Micro\Az1-3\Az1-3.c

Notes | Az1-3.c

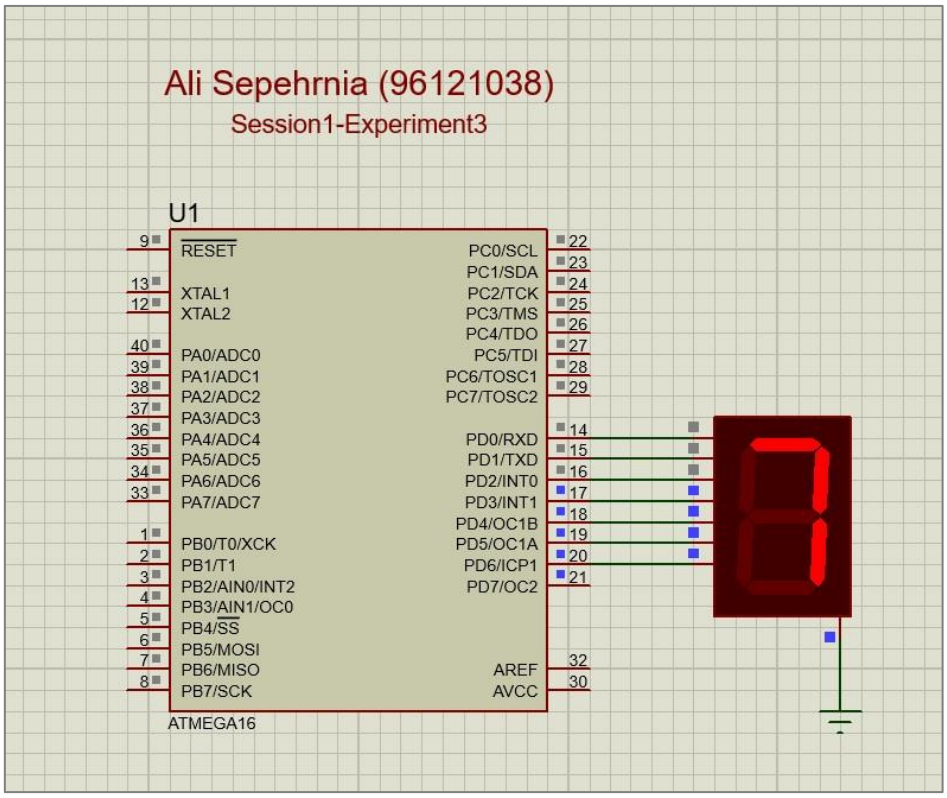
```
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
DDRD=0xff;
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

while (1)
      {
      // Place your code here
      for(i=0;i<=9;i++)
      {
        PORTD = sevenseg[i];
        delay_ms(500);
      }
      for(i=8;i>=1;i--)
      {
        PORTD = sevenseg[i];
        delay_ms(500);
      }
      }
```


Ali Sepehrnia (96121038)
Session1-Experiment3
U1
RESET
XTAL1
XTAL2
PA0/ADC0
PA1/ADC1
PA2/ADC2
PA3/ADC3
PA4/ADC4
PA5/ADC5
PA6/ADC6
PA7/ADC7
PB0/T0/XCK
PB1/T1
PB2/AIN0/INT2
PB3/AIN1/OC0
PB4/SS
PB5/MOSI
PB6/MISO
PB7/SCK
ATMEGA16
PC0/SCL
PC1/SDA
PC2/TCK
PC3/TMS
PC4/TDO
PC5/TDI
PC6/TOSC1
PC7/TOSC2
PD0/RXD
PD1/TXD
PD2/INT0
PD3/INT1
PD4/OC1B
PD5/OC1A
PD6/ICP1
PD7/OC2
AREF
AVCC